

Chemical Reactions in Geant4-DNA

A beginner's guide to the reaction types and how to build them

2026-09-02

1. Where reactions live in a Geant4-DNA simulation
2. The mental model: tracked species vs. bulk species
3. Type 1 — Unimolecular decay (water dissociation channels)
 - What it is
 - How you build it
 - What a beginner actually does
4. Type 2 — Bimolecular reactions `A + B -> products`
 - The data object
 - Where the rate constant comes from
 - Reaction "types" (`SetReactionType`)
 - Building type 2 reactions — from a MACRO
 - Building type 2 reactions — from CODE
5. Type 3 — Scavenger / bulk reactions `A + [X] -> products`
 - Step A — declare the bulk composition (`G4VChemistryWorld`)
 - Step B — wire the scavenger material into the scheduler
 - Step C — the reactions (`G4DNAScavengerProcess`)
 - Requirements / gotchas for type 3
 - Can type 3 be done from a macro?
6. Type 4 — Electron-hole recombination, and solvation
7. How the time-step model consumes reactions
8. Decision tree: "I want to add reaction X — where does it go?"
9. Worked example: add dissolved-O2 scavenging
10. Common beginner pitfalls
11. Where to look
12. Cheat sheet

1. Where reactions live in a Geant4-DNA simulation

A Geant4-DNA simulation of water radiolysis has three stages:

PHYSICAL (< 1 fs)	PHYSICO-CHEMICAL (1 fs – 1 ps)	CHEMICAL (1 ps – μ s ... s)
ionisation / excitation of water by the primary beam	-> excited/ionised water breaks up; electron thermalises and solvates (e-aq)	-> diffusion of radicals (e-aq, OH, H, H ₂ O ₂ ...) + REACTIONS between them

Everything this tutorial is about happens in the **chemical stage**: molecules diffuse (Brownian motion or an independent-reaction-time model) and, when they meet, they **react**.

A “reaction” in Geant4-DNA is always one of a small number of *kinds*, and each kind is built with a different framework class. Knowing which kind you have tells you exactly where the code goes.

kind	example	framework object	where you build it
1. Unimolecular decay	H ₂ O* -> OH + H	G4MolecularDissociationChannel	ConstructDissociatio
2. Bimolecular	OH + OH -> H ₂ O ₂	G4DNAMolecularReactionData in the reaction table	ConstructReactionTab /chem/reaction/add
3. Scavenger / bulk	e-aq + O ₂ (bulk) -> O ₂ ⁻	G4DNAScavengerProcess + G4DNAScavengerMaterial	ConstructProcess() + G4VChemistryWorld
4. Electron-hole recombination	H ₂ O ⁺ + e ⁻ -> H ₂ O*	G4DNAElectronHoleRecombination	ConstructProcess()
(solvation)	e ⁻ -> e ⁻ aq	G4DNAElectronSolvation	ConstructProcess()

The rest of this document takes them one at a time.

2. The mental model: tracked species vs. bulk species

Before the reaction types, one distinction that explains almost everything.

Tracked species — `e-aq`, `°OH`, `H`, `H2O2`, `H2O`, `O2^-` ...

- Each molecule is an individual `G4Track` with a **position**.
- It **diffuses** (diffusion coefficient `D > 0`).
- Two tracked species react when they get close enough (SBS) or at a sampled encounter time (IRT).
- Counted by `G4MoleculeCounter` → this is what `Species.root` / G-values report.

Bulk (background) species — dissolved `O2`, the `H3O+` / `OH-` fixed by pH, bulk water itself, `NO2^-`, `Fe2+` ...

- **Not** individual tracks. Represented as a single **scalar concentration**.
- Does **not** diffuse (`D = 0`). It is everywhere, uniformly.
- A tracked radical reacts with it at a **pseudo-first-order** rate `k · [X]`, anywhere in the box.
- Stored in `G4VChemistryWorld::fpChemicalComponent` (a `species` → `molarity` map) and counted by `G4DNAScavengerMaterial`.

In the molecule table the bulk versions of water species carry a `(B)` suffix:

`H2O(B)`, `H3Op(B)`, `OHm(B)`. They are created by

`G4ChemDissociationChannels_option1::ConstructMolecule()` with `D = 0`.

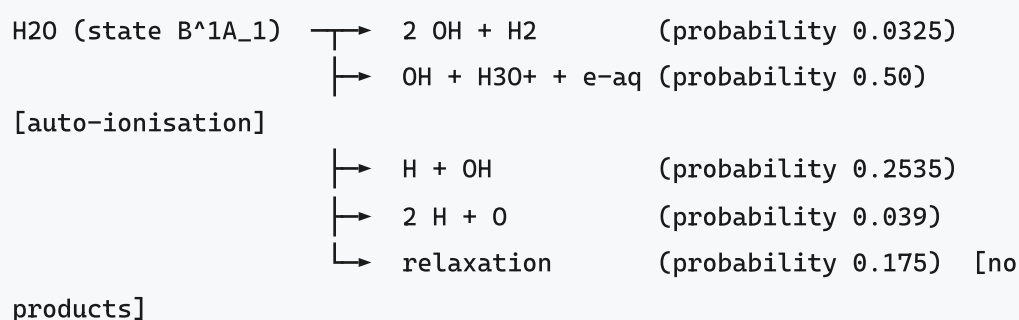
Why bother? A real solution has $\sim 10^{22}$ `O2` molecules per mL. You cannot track them. Modelling a solute as a concentration is $O(1)$ per step instead of $O(N^2)$. That is the entire reason "type 3" exists.

A configuration can be **both**: `O2` is a tracked species (a product of `H2O + H2O -> H2O2 + O2`) and a bulk species (dissolved oxygen you set with a macro command).

3. Type 1 — Unimolecular decay (water dissociation channels)

What it is

After the physical stage, water is left in an excited or ionised electronic state. Those states are unstable and **decay** into radicals along several competing **channels**, each with a branching ratio:



This is *not* a bimolecular reaction — there is no partner. It never appears in the reaction table.

How you build it

```
// one channel of one water electronic state
auto* decay = new G4MolecularDissociationChannel("B1A1_dissoc");
decay->AddProduct(OH); // a G4MolecularConfiguration*
decay->AddProduct(OH);
decay->AddProduct(H2);
decay->SetProbability(0.0325); // branching ratio (must sum to ~1
    per state)
decay->SetDisplacementType(
    G4DNAWaterDissociationDisplacer::B1A1_DissociationDecay); // where
    products appear
// decay->SetEnergy(...); decay->SetDecayTime(...); //
    optional

// attach it to the water electronic state
G4H2O::Definition()->AddDecayChannel("B^1A_1", decay);
```

You do this inside your chemistry list's `ConstructDissociationChannels()` override. It is executed at run time by the `G4DNAMolecularDissociation` rest process on `G4H2O` (registered in `ConstructProcess()` with ordinal 1).

What a beginner actually does

Nothing. You almost never write these by hand. Delegate to the reference set:

```
void MyChemistryList::ConstructDissociationChannels() {  
    G4ChemDissociationChannels_option1::ConstructDissociationChannels();  
}
```

The stock `G4EmDNAChemistry_option*` constructors already do this. You only touch dissociation channels if you are doing research on the branching ratios themselves.

There is **no macro command** for dissociation channels.

4. Type 2 — Bimolecular reactions $A + B \rightarrow \text{products}$

This is the reaction type you will spend 95% of your time on: two diffusing radicals meet and react.

The data object

Every bimolecular reaction is one `G4DNAMolecularReactionData`:

```
#include "G4DNAMolecularReactionTable.hh"

// rate in M^-1 s^-1 -> multiply by the Geant4 unit factor
const G4double perMs = 1e-3 * m3 / (mole * s); // "dm^3 / (mol s)"

auto* d = new G4DNAMolecularReactionData(0.55e10 * perMs, "°OH", "°OH");
d->AddProduct("H2O2");
reactionTable->SetReaction(d); // reactionTable is the argument of
ConstructReactionTable()
```

Notes for beginners:

- Two constructors: by **name** ("°OH") or by **pointer** (`G4MoleculeTable::Instance()->GetConfiguration("°OH")`). Names must match the molecule table **exactly** — a typo gives a `nullptr` and a crash later, not a helpful error.
- **Both constructors already call `ComputeEffectiveRadius()`**. Pass the real rate constant; you do *not* need the `new ... (0., ...)` then `SetObservedRate...` dance (that pattern only exists because the macro parser reads the rate after constructing the object).
- **No products** = a valid reaction (e.g. $\text{H3O}^+ + \text{OH}^- \rightarrow (2 \text{ H2O})$ — water is not tracked). Just don't call `AddProduct`.
- A product literally named "H2O" is **dropped** by the macro parser (water is not counted); in code you *can* add it but it rarely matters.

Where the rate constant comes from

`ComputeEffectiveRadius()` inverts the Smoluchowski relation:

$$R_{\text{eff}} = \frac{k_{\text{obs}}}{4 \pi \cdot \sum D \cdot N_A} \quad (\sum D = D_A + D_B, \text{ or } D_A \text{ alone if } A == B)$$

- **SBS** then reacts two tracks if their separation $< R_{\text{eff}}$.
- **IRT** samples a reaction time from the first-passage distribution using R_{eff} and the separation.

So the diffusion coefficients of **A** and **B** directly change the geometry of the reaction. Set them with `/chem/species` or in `ConstructMolecule()`.

Reaction “types” (`SetReactionType`)

type	name	meaning	who uses it
0 (default)	totally diffusion-controlled (TDC)	every encounter reacts, $P = 1$	SBS + IRT
1	partially diffusion-controlled (PDC)	activation-limited: reacts with $P < 1$ at a contact radius $vdW_A + vdW_B$	IRT / IRT_syn only
6, 7, 8	acid–base equilibrium pair	forward/back reactions throttled toward equilibrium by <code>G4ChemEquilibrium</code>	scavenger context only (Section 5)

```
d->SetReactionType(1); // call AFTER reactants + rate (it reads their D and vdW radii)
```

Beginner rule: if you run **SBS** (the common default), leave every reaction at type 0. `G4DNASmoluchowskiReactionModel` ignores type 1 — you would get an inconsistent radius. Only set type 1 if you deliberately run IRT and have activation-limited kinetics. Types 6/7/8 are never set from a plain macro.

Building type 2 reactions — from a MACRO

This is the supported, no-C++ workflow (`chem2`, `chem6`, and the classic `dnachem-min/beam.in` all use it):

```

# use a stock chemistry constructor for molecules + dissociation
channels,
# but tell the manager NOT to build its hard-coded reaction set:
/chem/skipReactionsFromChemList

/run/initialize

# now define the whole reaction set:
/chem/reaction/UI                                # reset the (shared)
table
/chem/reaction/add H + H -> H2                    | Fix | 1.2e10 | 0
/chem/reaction/add e_aq + °OH -> OHm              | Fix | 2.95e10 | 0
/chem/reaction/add °OH + °OH -> H2O2              | Fix | 0.44e10 | 0
/chem/reaction/add H3Op + OHm -> H2O              | Fix | 1.43e11 | 0    # H2O
product dropped -> "no product"
/chem/reaction/print                              # sanity check

```

Command reference (`G4ReactionTableMessenger`):

command	meaning
<code>... \ Fix \ <k> \ <0\ 1></code>	fixed rate in $\text{M}^{-1}\text{s}^{-1}$; trailing 1 sets reaction type 1, anything else = type 0
<code>... \ Arr \ <A0> <E_R></code>	Arrhenius $k(T) = A0 \cdot \exp(E_R/T)$
<code>... \ Pol \ <P0 P1 P2 P3></code>	log-polynomial $k(T) = 10^{(\sum P_i/T^i)}$
<code>... \ Scale \ <T_K> <k></code>	scales with the temperature-dependent water diffusion coefficient
<code>/chem/reaction/UI</code>	reset the table (call once, before the <code>add</code> lines)
<code>/chem/reaction/new A B <k></code> <code>[products...]</code>	older “diffusion-controlled” add
<code>/chem/temperature <T> <unit></code>	rescale all rates for a new temperature

Ordering matters. `/chem/reaction/UI` and the `add` lines can come *after* `/run/initialize` (the manager rebuilds the table lazily), but do not put `/chem/reaction/UI` in a macro if your C++ chemistry list already defined reactions — it wipes them.

Building type 2 reactions — from CODE

Override `ConstructReactionTable` in a `G4VUserChemistryList` subclass:


```

void MyChemistryList::ConstructReactionTable(G4DNAMolecularReactionTable*
    table)
{
    auto conf = [](const G4String& n){ return G4MoleculeTable::Instance()-
        >GetConfiguration(n); };
    auto add = [&](const G4String& a, const G4String& b, G4double k_perMs,
        std::initializer_list<G4String> products) {
        auto* d = new G4DNAMolecularReactionData(k_perMs * (1e-
            3*m3/(mole*s)), a, b);
        for (auto& p : products) d->AddProduct(p);
        table->SetReaction(d);
    };

    add("H", "H", 1.2e10, {"H2"});
    add("e_aq", "°OH", 2.95e10, {"OHm"});
    add("°OH", "°OH", 0.44e10, {"H2O2"});
    add("H3Op", "OHm", 1.43e11, {});           // no product
}

```

`ConstructReactionTable()` runs **once, on the master thread**. Good for large sets, temperature scans, or reactions computed from data files.

5. Type 3 — Scavenger / bulk reactions $A + [X] \rightarrow$ products

A tracked radical reacts with a **dissolved solute at fixed concentration** (dissolved O₂, the pH ions, added scavengers). Three framework pieces:

G4VChemistryWorld	G4DNAScavengerMaterial
G4DNAScavengerProcess	
_____	_____

species -> molarity	species -> molecule COUNT
diffusing radical:	per
(you set it: pH, O ₂)	= floor($N_A \cdot c \cdot V$)
= $k \cdot [X]$	propensity
	(refilled each event)
Gillespie-samples a	
	reaction
TIME	

Step A — declare the bulk composition (G4VChemistryWorld)

There is **no stock concrete class** and **no stock messenger** for this — every project writes ~40 lines. Minimal shape:

```

class MyChemistryWorld : public G4VChemistryWorld {
    void ConstructChemistryBoundary() override {           // the diffusion
        box
        G4double h = 0.5*mm;
        fpChemistryBoundary = std::make_unique<G4DNABoundingBox>(
            std::initializer_list<G4double>{h,-h, h,-h, h,-h}); //
            {xhi,xlo,yhi,ylo,zhi,zlo}
    }
    void ConstructChemistryComponents() override {         // the
        concentrations
        auto* t = G4MoleculeTable::Instance();
        fpChemicalComponent[t->GetConfiguration("H2O(B)")] = 55.3 /
            (mole*liter);
        fpChemicalComponent[t->GetConfiguration("H3Op(B)")] = std::pow(10,-
            fpH) / (mole*liter);
        fpChemicalComponent[t->GetConfiguration("OHm(B)")] = std::pow(10,-
            (14-fpH)) / (mole*liter);
        fpChemicalComponent[t->GetConfiguration("O2")] = fO2molar;
            // dissolved O2
    }
    G4double fpH = 7, fO2molar = 0;
};

```

Add your own `G4GenericMessenger` for `/.../pH`, `/.../O2` (both `PreInit`) so the macro can drive it.

Step B — wire the scavenger material into the scheduler

```

// per worker thread, AFTER ConstructChemistryComponents() has run:
auto sc = std::make_unique<G4DNAScavengerMaterial>(chemWorld);
sc->SetCounterAgainstTime();           // enables the time
    history
G4Scheduler::Instance()->SetScavengerMaterial(std::move(sc));

```

Step C — the reactions (G4DNAScavengerProcess)

```
// one process per diffusing species that reacts with the bulk
auto* proc = new G4DNAScavengerProcess("scavenger", *chemWorld-
    >GetChemistryBoundary());

// e_aq + O2(bulk) -> O2^-      (bimolecular, rate x concentration at
    run time)
auto* d1 = new G4DNAMolecularReactionData(1.74e10 * (1e-3*m3/(mole*s)),
    e_aq, O2);
d1->AddProduct(O2m);
proc->SetReaction(e_aq, d1);      // 1st arg = the diffusing species;
    the bulk is the other reactant

// O2^- + H2O -> HO2 + OH-(bulk) (first-order "with water": rate is k *
    [H2O] already, units 1/s)
auto* d2 = new G4DNAMolecularReactionData(0.15 * 55.3 / s, O2m, H2O);
    // note "/ s", not the M^-1s^-1 factor
d2->AddProduct(HO2); d2->AddProduct(OHmB);
proc->SetReaction(O2m, d2);

// O2^- + H3O+(bulk) <=> HO2      (acid-base equilibrium)
auto* d3 = new G4DNAMolecularReactionData(4.78e10 * (1e-3*m3/(mole*s)),
    O2m, H3OpB);
d3->AddProduct(HO2);
d3->SetReactionType(6);          // equilibrium type -> throttled by
    G4ChemEquilibrium
proc->SetReaction(O2m, d3);

G4PhysicsListHelper::GetPhysicsListHelper()->RegisterProcess(proc,
    G4Electron_aq::Definition());
```

Do this in `ConstructProcess()`. **Rate units for type 3:**

reaction shape	rate you pass
A + X(B) where X has a concentration	$k \cdot (1e-3 \cdot m^3 / (mole \cdot s))$ — multiplied by [X] at run time
A + H2O (first order)	k / s or $k \cdot 55.3 / s$ — used directly, not multiplied

Requirements / gotchas for type 3

- `G4DNAScavengerProcess` needs a configuration literally named **"H2O"** (`G4MoleculeTable::Instance()->CreateConfiguration("H2O", G4H2O::Definition())` in `ConstructMolecule()`). `_option1` only makes **"H2O(B)"**.

- If `BuildPhysicsTable()` runs while the scheduler has **no** scavenger material, `fpScavengerMaterial` is null and the process **segfaults** on the first step (no null check). Create the material *before* physics tables are built — practically: at the end of `ConstructProcess()`, after `G4DNAChemistryManager::Instance()->Initialize()`.
- Bulk pools for `H2O` / `H3Op(B)` / `OHm(B)` are **held constant** (fixed pH). Only non-ion scavengers such as `O2` actually deplete.
- `SetReaction()` after the process is initialised → `FatalErrorInArgument`. Register everything during `ConstructProcess()`.

Can type 3 be done from a macro?

Not with stock commands. There is no messenger for `G4DNAScavengerProcess`, no messenger for `G4VChemistryWorld` concentrations, and `/chem/reaction/add` cannot set equilibrium types 6/7/8. The concentrations + the scavenger-process reactions are the irreducible C++ of a scavenger simulation.

6. Type 4 — Electron-hole recombination, and solvation

Two more chemical-stage processes, both registered in `ConstructProcess()`, both “free” from the stock constructors:

Electron-hole recombination — a sub-excitation electron recombines with its parent water cation instead of solvating:

```
G4H2O::Definition()->GetProcessManager()  
->AddRestProcess(new G4DNAElectronHoleRecombination(), 2); //  
    ordinal 2
```

The recombined water lands in the vibrational state `H2Ovib`, whose decay channels (`2 OH + H2`, `OH + H`, `2 H + O`, relaxation) are part of the type-1 dissociation set.

Electron solvation — the free electron thermalises into `e-aq`:

```
if (!G4ProcessTable::GetProcessTable()->FindProcess("e-  
_G4DNAElectronSolvation", "e-"))  
    ph->RegisterProcess(new G4DNAElectronSolvation("e-  
_G4DNAElectronSolvation"),  
        G4Electron::Definition());
```

Model selectable from a macro: `/process/dna/e-SolvationSubType Ritchie1994 (fast) / Meesungnoen2002 / Terrisol1990`. Not a “reaction”, but it sets the starting point for all `e-aq` chemistry.

7. How the time-step model consumes reactions

Set it with `/process/chem/TimeStepModel SBS | IRT | IRT_syn` (before `/run/initialize`).

model	class	how it uses type-2 reactions	how it uses type-3 (bulk) reactions
SBS	<code>G4DNAMolecularStepByStepModel</code> + <code>G4DNASmoluchowskiReactionModel</code>	react if two tracks are within <code>R_eff</code>	<code>G4DNAScavengerProcess</code>
IRT	<code>G4DNAMolecularIRTModel</code>	sample an encounter time per pair	reaction-table (B) entry <code>G4DNAGillespieDirectM</code>
IRT_syn	<code>G4DNAIndependentReactionTimeModel</code>	synchronised IRT	same as IRT

Consequence you will see in real code (UHDR, dnachem-min): a bulk reaction such as `e_aq + O2(B) -> O2^-` is often registered **twice** — once in the reaction table (for IRT) *and* once as a `G4DNAScavengerProcess` reaction (for SBS). Under SBS the table copy is inert (no O2 tracks to match); under IRT the scavenger process is inactive. Registering both makes the chemistry model-agnostic.

Register the time-step model in `ConstructTimeStepModel()`:

```
void MyChemistryList::ConstructTimeStepModel(G4DNAMolecularReactionTable*) {
    auto m = G4EmParameters::Instance()->GetTimeStepModel();
    if (m == G4ChemTimeStepModel::SBS) RegisterTimeStepModel(new
        G4DNAMolecularStepByStepModel(), 0);
    else if (m == G4ChemTimeStepModel::IRT_syn) RegisterTimeStepModel(new
        G4DNAIndependentReactionTimeModel(), 0);
    else /* IRT */ RegisterTimeStepModel(new
        G4DNAMolecularIRTModel(), 0);
}
```

8. Decision tree: “I want to add reaction X — where does it go?”

```
Is one reactant an unstable water state (H2O*, H2O+, H2Ovib)?
|
├─ YES → TYPE 1. A G4MolecularDissociationChannel in
ConstructDissociationChannels().
|
|   (You almost certainly should just reuse
G4ChemDissociationChannels_option1.)
|
└─ NO → Are BOTH reactants tracked, diffusing species (D > 0)?
      |
      └─ YES → TYPE 2. G4DNAMolecularReactionData in the reaction
table.
          |
          |   Macro: /chem/reaction/add A + B -> C | Fix | k | 0
          |   Code:  table->SetReaction(new
G4DNAMolecularReactionData(k, A, B));
          |
          └─ NO (one reactant is a fixed-concentration solute) →
              TYPE 3. G4DNAScavengerProcess + a G4VChemistryWorld
concentration.
              Needs C++.  If the reaction is "A + H2O", pass the rate
as k/s.
              If it is an acid-base pair, SetReactionType(6|7|8).
              Also add it to the reaction table (bare) so IRT sees
it.
```


9. Worked example: add dissolved-O2 scavenging

Starting from a pure-water simulation, add “water equilibrated with 21% O2”.

1. Molecules — already present in `_option1` (`O2`, `H02°`, `O2m`, `H02m`, `Om`, `O3m`).
Add the plain `"H2O"` config the scavenger process needs:

```
void MyChemistryList::ConstructMolecule() {  
    G4ChemDissociationChannels_option1::ConstructMolecule();  
    G4MoleculeTable::Instance()->CreateConfiguration("H2O",  
        G4H2O::Definition());  
}
```

2. Concentration — in `MyChemistryWorld::ConstructChemistryComponents()`:

```
// 21 % of a pure-O2 atmosphere, Henry constant kH = 0.0013 M  
fpChemicalComponent[t->GetConfiguration("O2")] = (21./100.) * 0.0013 /  
    (mole*liter);
```

3. Bulk-O2 reactions — reaction table (for IRT) *and* scavenger process (for SBS),
rates from the literature (identical to the UHDR example):

e_{aq}	+	O2	->	O2 ⁻	1.74e10 M ⁻¹ s ⁻¹
H	+	O2	->	H02	2.1e10
O ⁻	+	O2	->	O3 ⁻	3.7e9

4. Fate of the products — the O2⁻ / HO2 / HO2⁻ radical network in the
reaction table (18 reactions, all type 0), plus their acid–base equilibria as
scavenger-process reactions (`O2- + H3O+(B)` type 6, `HO2 + H2O` type 6, `HO2- + H2O` type 7, `O- + H2O` type 8, ...).

5. Wire the scavenger material at the end of `ConstructProcess()`.

Result: dissolved O2 scavenges the reducing species — at 1 μs, `e-aq` and `H` are
driven to zero and converted to superoxide `O2-` and `HO2`. That is the “oxygen
effect”.

10. Common beginner pitfalls

symptom	cause	fix
reaction silently never fires	species name typo ("OH" vs "°OH" vs "OHm")	<code>/chem/reaction/print</code> ; check names against <code>/chem/PrintSpeciesTable</code>
<code>/chem/reaction/add</code> lines ignored	issued before <code>/run/initialize</code> and no <code>skipReactionsFromChemList</code> , or the C++ list overwrote them	use <code>/chem/skipReactionsFromChemList</code> don't mix macro + C++ reactions
segfault at first chemistry step	<code>G4DNAScavengerProcess</code> with no <code>G4DNAScavengerMaterial</code> set	create the material before physics tables build
<code>FatalException</code> : total diffusion coefficient is null	a reaction between two <code>D = 0</code> species (two (B) species) put in the reaction table	bulk × bulk only works through the scavenger path
type-1 reaction behaves oddly under SBS	<code>SetReactionType(1)</code> + SBS	keep type 0 for SBS
O2 set but nothing changes	concentration in the <code>G4VChemistryWorld</code> map is 0, or no scavenger reactions registered	check <code>ConstructChemistryComponents()</code> ran and populated the map
results differ serial vs MT	expected — different RNG streams; compare merged G-values, not per-event	



11. Where to look

Source (`source/processes/electromagnetic/dna/`)

file	what
<code>utils/G4DNAMolecularReactionTable.{hh,cc}</code>	<code>G4DNAMolecularReaction</code> + the table; <code>ComputeEffectiveRadius</code> <code>SetReactionType</code>
<code>utils/G4ReactionTableMessenger.cc</code>	the <code>/chem/reaction/*</code> command parser
<code>molecules/management/G4MoleculeTableMessenger.cc</code>	<code>/chem/species</code>
<code>molecules/management/G4MolecularDissociationChannel.hh</code>	type-1 decay channels
<code>processes/G4DNAScavengerProcess.{hh,cc}</code>	type-3 process
<code>utils/G4DNAScavengerMaterial.{hh,cc}</code>	the bulk-concentration bookkeeping
<code>utils/G4VChemistryWorld.hh</code>	the concentration map + boundary
<code>models/G4DNASmoluchowskiReactionModel.cc</code> , <code>G4DiffusionControlledReactionModel.cc</code>	how <code>R_eff</code> is consumed (IRT)
<code>physics_lists/.../G4ChemDissociationChannels_option1.cc</code>	the reference molecule + dissociation set

Examples (`examples/extended/medical/dna/`)

example	reaction style to learn from
<code>chem2</code> , <code>chem6</code>	type 2 fully in a macro (<code>/chem/reaction/add</code>)
<code>chem4</code>	type 2 programmatically
<code>UHDR</code>	all types: builder classes for the table, <code>G4DNAScavengerProcess</code> for the bulk, a <code>G4VChemistryWorld</code> subclass, pH / O2 messenger

12. Cheat sheet

```
TYPE 1  water decay          H2O* -> radicals
      build: G4MolecularDissociationChannel +
H2O::AddDecayChannel(state, ch)
      in:   ConstructDissociationChannels()  (just call _option1)
      run:   G4DNAMolecularDissociation rest process
      macro: none

TYPE 2  bimolecular          A + B -> products      (A, B both tracked, D >
0)
      build: new G4DNAMolecularReactionData(k, A, B); AddProduct(...);
table->SetReaction(d)
      in:   ConstructReactionTable()   OR   /chem/reaction/add
      rate: k * (1e-3*m3/(mole*s))      [M^-1 s^-1]
      type: 0 = diffusion-controlled (use for SBS); 1 =
activation-limited (IRT only)
      macro: /chem/skipReactionsFromChemList
             /chem/reaction/UI
             /chem/reaction/add A + B -> C + D | Fix | k | 0

TYPE 3  scavenger / bulk    A + X(bulk) -> products      (X = fixed
concentration)
      build: G4VChemistryWorld (concentrations)
             + G4DNAScavengerMaterial ->
G4Scheduler::SetScavengerMaterial
             + G4DNAScavengerProcess::SetReaction(mol, data) ; ph-
>RegisterProcess(proc, def)
      in:   ConstructChemistryComponents() + ConstructProcess()
      rate: A + X(B) : k * (1e-3*m3/(mole*s))      (x concentration at
run time)
             A + H2O : k / s   or   k * 55.3 / s   (first order, used
directly)
      type: 6/7/8 for acid-base equilibrium pairs
      macro: not possible with stock commands
      note: also add the reaction (bare) to the reaction table so IRT
sees it
      need: a "H2O" configuration must exist

TYPE 4  e-/hole recomb.     H2O+ + e- -> H2O(vib)
```

```
build: H2O::ProcessManager->AddRestProcess(new
G4DNAElectronHoleRecombination(), 2)
    solvation: G4DNAElectronSolvation on e- ; /process/dna/e-
SolvationSubType <model>

TIME-STEP MODEL    /process/chem/TimeStepModel  SBS | IRT | IRT_syn
    SBS  -> G4DNAScavengerProcess handles bulk
    IRT  -> reaction-table (B) entries handle bulk
```

Grounded in Geant4 11.4.1 source. Prepared for the `dnachem-min` project; companion to the local knowledge base (`chemistry-stage.md`, `dna-processes.md`, `uhdr-specifics.md`, `gotchas.md`).